Deepfake Detection System

Findings & Documentation

Balaiah Tarun (2022132) Md Kaif (2022289) Nishant Kumar Singh (2022327)
Vaishvi Verma (2022609)

Team Vincenzo
Computer Vision Course Project
IIIT-Delhi

March 2026

1 Project Overview

A web-based forensic tool that detects AI-manipulated (deepfake) facial images using an EfficientNet-B4 CNN, trained on the FaceForensics++ dataset. The system provides binary classification (TEXTSCREAL/FAKE) with confidence scores and Grad-CAM localization heatmaps.

1.1 Tech Stack

Component	Technology
Frontend	Next.js 15, Tailwind CSS
Backend	FastAPI (Python)
Model	EfficientNet-B4 (via `timm`)
Face Detection (Preprocessing)	MTCNN (facenet-pytorch)
Face Detection (Inference)	OpenCV Haar Cascade
Experiment Tracking	Weights & Biases (W&B)
Training Infrastructure	IIITD Precision Cluster (SLURM)

2 Dataset

FaceForensics++ (C23 compression)
Source: Kaggle (`xdxd003/ff-c23`), Format: MP4 videos, Size: ∼17.9 GB

Category	Manipulation Type	Videos
Original (Real)	—	1,000
Deepfakes	Face swap	1,000
Face2Face	Expression transfer	1,000
FaceSwap	Face swap	1,000
FaceShifter	Face swap	1,000
NeuralTextures	Neural rendering	1,000
DeepFakeDetection	Face swap	1,000

Table 1: FaceForensics++ dataset composition. Only `Deepfakes/` and `original/` were used for training in the current iteration.

3 Preprocessing Pipeline

The preprocessing pipeline (`backend/preprocess.py`) extracts face crops from raw videos:

1. Read MP4 videos using OpenCV
2. Extract every 10th frame (`FRAME_INTERVAL = 10`)
3. Run MTCNN face detection (PyTorch, GPU-accelerated) on each frame
4. Crop detected face with 20% margin around bounding box
5. Resize to 224×224 pixels
6. Save as PNG to `faces/real/` or `faces/fake/` (organized by video name)

Parameter	**Value**
Frame interval	Every 10th frame
Face size	224×224
Bounding box margin	20%
Min detection confidence	0.90
Max faces per video	20

Table 2: Preprocessing configuration.

Split	**Frames**
Train (70%)	28,020
Val (15%)	6,017
Test (15%)	6,025
Total	**40,062**

Table 3: Dataset split sizes.

3.1 Key Design Decision: Video-Level Split

The train/val/test split is performed at the **video level**, not the frame level. This prevents data leakage — frames from the same video never appear in both training and test sets.

4 Data Augmentation

Applied during training only (not validation/test):

Augmentation	**Parameters**
Random horizontal flip	50% probability
Color jitter	brightness=0.3, contrast=0.3, saturation=0.2, hue=0.05
Random rotation	up to 10 degrees
JPEG compression simulation	quality 50–95, 50% probability
ImageNet normalization	mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]

Table 4: Training augmentations. JPEG compression simulation is particularly important for FaceForensics++ generalization, as the dataset uses C23 (quality 23) compression.

5 Model Architecture

EfficientNet-B4 (pretrained on ImageNet, loaded via `timm`):

- Classifier head replaced with: `Dropout(0.4)` → `Linear(in_features, 1)`
- Single sigmoid output for binary classification
- Loss function: `BCEWithLogitsLoss`

5.1 Training Strategy: Freeze → Unfreeze

Phase	Epochs	What's Trained	Learning Rate
Head-only	1–3	Classifier head only	1×10^{-3} (10× base)
Full fine-tune	4–20	All parameters	1×10^{-4} with cosine annealing

Table 5: Two-phase training strategy.

5.2 Hyperparameters

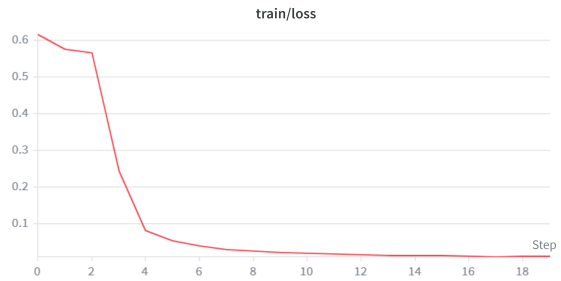
Parameter	Value
Optimizer	AdamW
Base learning rate	1×10^{-4}
Weight decay	1×10^{-4}
Batch size	32
Epochs	20
Scheduler	CosineAnnealingLR ($\eta_{\min} = 1 \times 10^{-6}$)
Dropout	0.4

Table 6: Training hyperparameters.

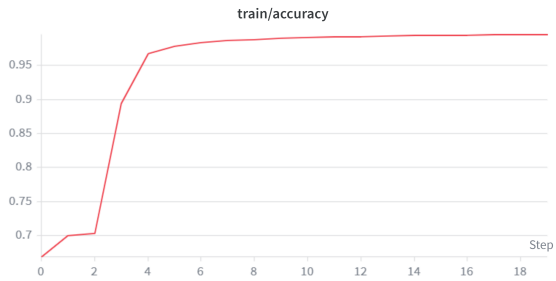
6 Training Results

Trained on: NVIDIA A100-SXM4-40GB (IIITD Precision Cluster, gpu04)
Training time: ∼1.5 hours (20 epochs)

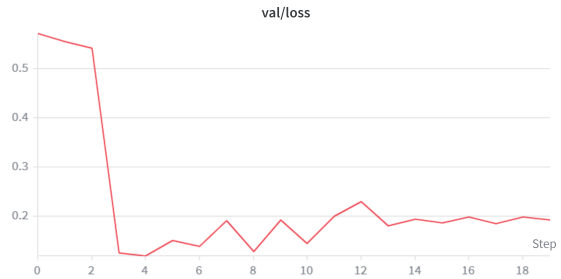
6.1 Training Curves



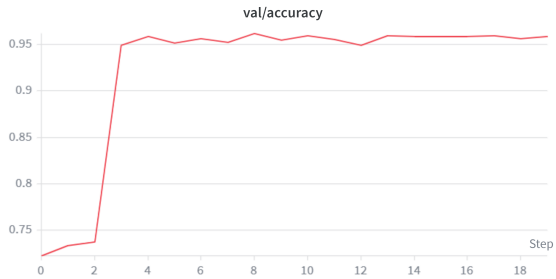
(a) Training Loss



(b) Training Accuracy



(c) Validation Loss



(d) Validation Accuracy

Figure 1: Training curves logged via Weights & Biases. Note the sharp improvement at epoch 3→4 when the backbone is unfrozen for full fine-tuning.

6.2 Final Metrics

Metric	Train	Validation	Test
Loss	0.0110	0.1924	0.1207
Accuracy	99.5%	95.9%	**96.1%**
F1-Score	99.5%	95.7%	**96.3%**
AUC-ROC	99.99%	99.3%	**99.5%**

Table 7: Final evaluation metrics across splits.

6.3 Observations

- **Strong performance:** 96.1% test accuracy and 99.5% AUC on held-out test set.
- **Slight overfitting:** Train accuracy (99.5%) vs test (96.1%) shows a ∼3.4% gap, which is expected and acceptable for this dataset size.
- **High AUC:** 99.5% AUC indicates excellent discriminative ability — the model ranks fake images higher than real ones almost perfectly.
- **Convergence:** Loss stabilized around epoch 15, with diminishing returns in later epochs.
- **Freeze→Unfreeze impact:** The sharp jump at epoch 3→4 validates the two-phase training strategy. Head-only training provides a warm start before full fine-tuning.

7 Inference Pipeline

The inference pipeline (`backend/model.py`) processes images as follows:

1. **Face validation:** OpenCV Haar Cascade checks for human face presence (rejects non-face images such as cats, landscapes, etc.)
2. **Preprocessing:** Resize to 224×224, normalize with ImageNet statistics
3. **Grad-CAM:** Hooks on the last convolutional layer (`conv_head`) capture feature maps and gradients to generate a localization heatmap
4. **Inference:** Forward pass through EfficientNet-B4, sigmoid on logit
5. **Output:** Label (REAL/FAKE), confidence score, individual probabilities, and Grad-CAM heatmap overlay

7.1 Grad-CAM Localization

Grad-CAM (Gradient-weighted Class Activation Mapping) generates visual explanations for the model's predictions by:

- Computing gradients of the output logit with respect to the final convolutional layer's feature maps
- Global average pooling the gradients to obtain channel-wise importance weights
- Computing a weighted sum of the feature maps, followed by ReLU activation
- Upsampling the resulting heatmap and overlaying it on the original image

This highlights the facial regions the model considers most important for its REAL/FAKE decision, providing interpretability and approximate manipulation localization.

7.2 Inference Speed

- CPU (laptop): ∼100–200ms per image
- GPU: ∼10–20ms per image

8 Limitations & Known Issues

8.1 Model Limitations

1. **Only trained on Deepfakes manipulation type** — the model has only seen face-swap deepfakes from the `Deepfakes/` folder. It has not been trained on Face2Face, FaceSwap, FaceShifter, or NeuralTextures manipulations.
2. **GAN-generated faces not detected** — images from tools like "thispersondoesnotexist.com" (StyleGAN) are a different type of manipulation and may not be reliably detected.
3. **Compression sensitivity** — trained on C23 compression; performance may degrade on heavily compressed (social media) or uncompressed images.
4. **Single-face assumption** — the inference pipeline processes the entire image, not individual face crops. Multi-face images may produce unreliable results.

8.2 Infrastructure Findings

1. **ONNX Runtime GPU incompatible with Precision cluster** — InsightFace (`onnxruntime-gpu`) could not be made to work due to cuDNN version mismatches across nodes. Solved by switching to facenet-pytorch (MTCNN) which uses PyTorch CUDA directly.
2. **MIG partitions (gpu01 short queue) cause cuDNN init failures** — use full GPU allocations on medium/long queues instead.
3. **cuDNN version conflict** — PyTorch requires cuDNN 8, `onnxruntime-gpu` 1.18+ requires cuDNN 9. Installing both simultaneously breaks one or the other.

9 Future Improvements

1. **Train on all manipulation types** — include Face2Face, FaceSwap, FaceShifter, NeuralTextures for better generalization
2. **Add DFFD and ForgeryNet datasets** — cross-dataset training for robustness
3. **Enhanced localization** — train a segmentation head for pixel-level manipulation masks instead of relying on Grad-CAM approximations
4. **Face cropping at inference** — use MTCNN to crop the face before running the classifier, matching the training pipeline
5. **Cross-dataset evaluation** — test on unseen datasets to measure generalization
6. **Model compression** — distillation or pruning for faster inference on edge devices